Consistency by Construction: Keeping a Multi-Project Agentic Ecosystem Correct as Projects and Agents Grow

Mostafa A. M. Sayed Adawy
[Affiliation to be confirmed by the author]
e-mail to be confirmed

***Abstract*—As agentic systems grow from single tasks into ecosystems of many projects, each with its own team of LLM agents, the hard problem becomes keeping the whole system consistent. Files must land where they belong, no project may depend on or overwrite another, there must be one source of truth, and no "done" may be false. We present Odin, a multi-project agentic ecosystem built and operated by one engineer with agents. Its eleven features were each forced by a concrete failure, which we reconstruct from the system's own logs. Consistency is enforced *by construction*: one creation door checks placement and owner scope on every write, a database is the only registry, and an audit checks and heals at rest. On an isolated copy of the production system, we measure with ratios rather than absolute times. Through the door, the violation rate stayed at 0 for 1, 2, 5 and 10 concurrent projects (up to 360 concurrent writes by 30 agents). Without the door it was 22–31%, and 11% of writes landed in other projects and could not be healed afterwards. With real LLM agents, 52 of 54 tasks succeeded and no agent wrote into another project. But files written with the agents' own tools bypass the door: correct placement fell from 67% to 17% as projects were added. The audit flagged every stray path. The owner's own lifecycle and adversarial tests passed 3/3 runs, and the integration hub delivered every callback under 20% injected failures.**

I. Introduction

Agentic systems built from large language models (LLMs) are moving from single tasks to long-lived *ecosystems*: many projects, each with its own team of agents, sharing scripts, credentials, infrastructure and a common workspace. In this setting the hardest problem is not whether one agent can solve one task, but whether the *whole system stays consistent* as projects and agents are added: files written where they belong, no project silently depending on or overwriting another, one source of truth for what exists, and no "done" that is not true. Recent analyses of multi-agent frameworks report that most failures stem from organisational design, misalignment between agents and missing verification rather than from model capability [1].

This paper reports on Odin, a multi-project agentic ecosystem that its builder, a single engineer working with agents, developed and operates for real projects: e-commerce operations, CRM and e-mail platforms, fulfilment, continuous security monitoring, meeting intelligence and software delivery. Odin started as a learning exercise. A widely used multi-agent framework was judged unsuitable for consistent, safe use; the first project worked; the second project revealed *interference between projects*. Every subsequent component was forced by a concrete failure (Section II).

Central claim: Consistency and correctness can be made to *hold as the system scales* by construction: one creation door that validates placement and owner scope on every write, a database that is the only registry, and an audit that checks and heals at rest what bypassed the door.

Contributions:

1) An architecture of eleven features for multi-project agentic ecosystems, each traced to the failure that forced it, with its origin in established engineering practice (component reuse, middleware routing, message queues, webhooks, change-based deployment, conformance checking).
2) A failure timeline reconstructed from the system's own logs, documents and history (Table I).
3) A measurement study on an isolated copy of the production system, reporting ratios rather than absolute times: consistency at 1, 2, 5 and 10 concurrent projects with and without the creation door, real LLM agents running concurrently, the owner's own lifecycle test, and the queue, callback and live-sync behaviour of the integration hub.

II. Origin: Each Failure Forced a Component

Odin began as a learning exercise. Its builder first evaluated a widely used multi-agent framework, judged it unsuitable for consistent and safe use, and deliberately chose a lean, deterministic dispatcher instead of a swarm (decision log, 21 June). The architecture was then grown one failure at a time. Table I lists every failure for which we found evidence in the system's own decision log, audit log, middleware-correction log, design documents or version history. Each row names the component that the failure forced into existence. Several components are independent re-discoveries of known patterns (Section VIII); we cite those patterns rather than claim them.

III. Architecture

Odin is a *multi-project agentic ecosystem*. A company-level CTO agent reasons over projects; each project has its own CTO and members (e.g., Fullstack, QA, DevOps, Fulfillment, Email, and twelve security agents); members work back and forth with the CTO, which re-plans and re-assigns when a

TABLE I
FAILURE TIMELINE (2026). EVIDENCE PATHS ARE RELATIVE TO THE SYSTEM'S REPOSITORY. ONLY FAILURES WITH EVIDENCE ARE LISTED.

Date	Failure	Evidence	Component it forced
06-21	A widely used multi-agent framework was judged unsuitable for consistent, safe use at scale	decision log ("reject swarm scale, self-evolving rules")	Lean deterministic dispatcher, explicitly not a swarm; later the truth gate
06-21	The "brain" ran on a small local model instead of the intended coding agent	decision log ("CORRECTION")	The Claude Code CLI as the engine; the local model only as an offline fallback
06-22	An owner's "approved" did not attach to the engine's own pending proposal	decision log	Approval gate bound to the engine's session memory
06-22	Requests stopped mid-way: the dev reloader killed workers because the server wrote its own data files	decision log ("ROOT CAUSE")	Runtime data separated from watched code
06-22	Keyword routing sent a folder named `*-frontend` to the Frontend agent	decision log (two entries)	Semantic router over a deterministic keyword floor (F2)
06-23	Owner's end-to-end run: 33 pass, 1 fail (the planner hallucinated agents from "no backend")	decision log (FIX)	Negation-aware team inference; later the scoped organisation
06-23	The engine could read and run but not *safely* sync local code to the VM	decision log (NOTE)	Mapping-file deploy flow and smart CI/CD (F9, F10)
06-21	Dynamic home IP: SSH lock-out after every IP change	decision log; firewall-rotation module	Dynamic-IP firewall: one /32 rule, replaced, never stacked (F8)
07-04	Web and mobile could not reach the desktop without a static IP	isolation-state document	Anchor point with a pulled queue and callbacks (F5, F6)
07-04	Live sync broke across devices: the Anchor point kept its mirror in memory, so every restart or offline period wiped it	isolation-state document ("NEW ROOT CAUSE"); scope sequencer	Durable mirror, per-scope sequence numbers, stale flag (F4)
07-16	**Second-project interference**: files landed in the wrong place, the CTO duplicated agents it already had, rules were ignored	structuring plan ("Why this kills the failures you keep hitting")	One creation door, a schema, the database as the only registry (F3)
07-17–30	Measured violations, caught and healed: 18 misplaced, 16 duplicates, 15 foreign references, 5 external writes, 3 parallel registries; 33 wrong-folder writes redirected	audit log; middleware-correction log	Audit layer (F11); duplicates promoted to the shared library (F1)
07-19	First run of a 4-new-project journey: 0/4; five root causes (agents missing from cards, incomplete delete, one-directional audit, copies instead of reuse)	phase-7 results report	Bidirectional audit, clean delete, pointer-plus-delta reuse. Second run: 4/4
07-30	Connector keys were tracked in version control	commit history	Secrets kept out of git

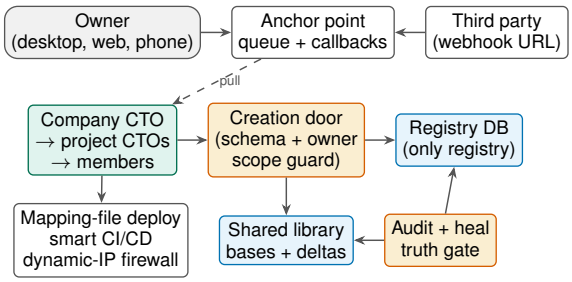


Fig. 1. Odin at a glance. Every created artefact passes one door; the audit checks what bypasses it.

result is insufficient. The engine is a coding agent (the Claude Code CLI) running in the project with terminal tools; Odin adds the organisation, the safety gates and the consistency machinery around it (Fig. 1). We describe the eleven features the owner identified, each with its origin.

F1 Component reuse (origin: React/React Native DRY): Shared scripts are treated as components and per-project deltas as their props [2], [3]: a total match points to the shared base, a near match stores only a small delta that references the base, and a different job composes a different agent. This is the same idea as delta-oriented product lines [4], applied to the operational scripts that agents run.

F2 Delta plus middleware semantic router (origin: backend middleware): A middleware routes each request to an existing base (reuse), a base plus delta, or new work. A deterministic keyword floor sits under a local semantic classifier so that safety-relevant routing never depends on the model alone.

F3 Shared-script library with a context-indexed database: When a new project is created, the middleware checks the library and the database against the project type and builds the requirement list; only missing capabilities are built. The CTO asks the owner "we already have this under *key* — use it?". The database is the *only* registry: a parallel JSON registry is itself a violation. Every create, update and delete passes one *creation door* that validates placement against a canonical schema, redirects a misplaced file to its correct slot, prevents a project from writing into another project's folder (owner-scope guard), and writes the file and its database row together.

F4 WhatsApp-style live sync: A change on any device propagates to all others. The Anchor point keeps a *durable* mirror, every message in a scope gets a monotonic sequence number assigned by one source, clients pull deltas by sequence (a missing message is a visible gap), and a stale flag shows when the desktop is offline (Table I, 07-04).

F5 Queue at the Anchor point (origin: RabbitMQ): An external requester with a secure key submits a request; it is stored as *queued*. When the owner's machine is connected it pulls by status, prioritises, processes, publishes and writes the status back. Pulling avoids any inbound connection to the owner's machine.

F6 Callback / return URL (origin: Stripe): The Anchor point posts the result to the requester's webhook and retries until it is acknowledged, so the contract is two-way: client-to-

server and server-to-client [5].

F7 Cross-project agent reuse: Agents live in an organisation-wide roster in the database; a project *links* existing agents (HubSpot, Veeqo, Klaviyo, fulfilmentand several security projects share agents) instead of cloning them.

F8 Dynamic-IP firewall (a feature): For operators without a static IP, deployment rotates the VM firewall and cloud security rule to the operator's current /32 address, replacing (never stacking) the previous one, so deployment stays possible without leaving a port open.

F9 Smart mapping-file deploy flow: A mapping file detects changed files by date and asks before updating; seed and database files are never touched automatically; a database update or rebase is asked separately, even on staging; after restart the flow verifies that real endpoints return real results and that newly seeded values are present.

F10 Smart CI/CD: Deterministic deployment "hands" are paired with an AI "brain" that probes the failing area (migration, build, ports, service, disk), consults a memory of learned faults, applies one safe fix and re-runs to verify, stopping for failures only a human can resolve.

F11 Audit layer: After a task, an audit checks that created files obey the shared/delta structure and sit in the folder declared for them, that no project references another project's private files, that the registry mirrors the disk in both directions, and heals what it can (move, promote a duplicate to the shared library, re-point a reference, re-index). A separate *truth gate* rejects any "done" claim that is not backed by evidence.

Also in the system: On-device Whisper speech-to-text and text-to-speech in English and Arabic; a Zoom meeting pipeline that turns live audio into a structured report; and phone and web control through the Anchor point. Connectors to SaaS platforms are plain API calls and are not a contribution.

IV. Evaluation Design

Our central claim is that *consistency and correctness hold as the system scales across projects and agents*. We ask:

- **RQ1** Does the creation door keep the workspace consistent as the number of concurrently active projects grows from 1 to 10, compared with agents writing directly?
- **RQ2** With real LLM agents running concurrently, how often do they write outside their own project or folder, does the audit catch it, and do tasks succeed?
- **RQ3** Does the owner's own end-to-end lifecycle test (create four projects of different shapes, monitor, delete) return the system to its exact baseline?
- **RQ4** How do the Anchor point's queue, callbacks and live sync behave: where does the time go, are callbacks delivered under failures, and does the mirror survive restarts?

Setting and isolation: All experiments ran on a fresh, isolated copy of the production system with no credentials (all keys blank), every reference to the production folder rewritten, distinct ports, scheduled tasks disabled, and shims that block cloud and remote tools (Azure CLI, SSH, rsync, sshpass) for every process, including tools invoked by agents. The registry database is a private PostgreSQL cluster restored from a read-only dump of the production registry; the production database was only read. Every trial runs in a throw-away root with a freshly restored database. The machine is a MacBook Pro with an Apple M4 Pro (12 cores, 24 GB RAM, macOS 26.5). The owner's production instance and its scheduled security agents kept running; we record their CPU load at the start and end of each experiment and report *ratios* instead of absolute times, together with the CPU time and memory of the specific process tree under test.

RQ1 workload: $N \in \{1, 2, 5, 10\}$ projects, three agents each, all writing concurrently (12 writes per agent). The write mix follows the violation classes recorded in the system's history: 60% valid, 15% wrong sub-folder, 5% project-root files, 10% writes into *another* project, 5% duplicated utilities, 5% references to another project's private file. Condition DOOR sends every write through the real middleware; condition BYPASS writes to the requested path. We then run the real audit and heal. Five seeds per cell; 40 trials.

RQ2 workload: $N \in \{1, 2, 5, 10\}$ projects are created through the system's HTTP API with a Fullstack and a QA member, and all N team orchestrations are started at once. Each project gets a small, verifiable task (a Python module and a plain-assert test). We measure, from a content hash of the whole root before and after, files changed inside the own project, inside another project, and outside any project; the audit's findings; whether the module exists and *its test passes when we run it*; cost and tokens from the CLI's own accounting; and the share of wall time during which at least one model call was in flight. The model was fixed to a small model (`haiku`) for cost.

RQ3: The owner's `phase7_lifecycle.py`, unmodified, three repetitions.

RQ4: Over real HTTP against the Anchor point's stand-in server: third-party clients submit M requests (50 with one client, 200 with four), a poller pulls, processes (synthetic, zero time, to isolate overhead) and publishes; a local receiver acknowledges callbacks, optionally failing 20% of them. Each request is time-tagged at *in, queued, pulled, processed, published* and *callback*. For live sync we push 20 pool updates and time push-to-visible, then restart the Anchor point and check the mirror. Three repetitions.

Regression suite: A suite of 18 checks (isolation invariants, compilation, live endpoints returning real data, the door's placement and owner-scope rules, the audit's detection classes, router determinism, the queue contract, and sequence contiguity) was baselined and re-run after every change to the isolated copy; it never regressed.

V. Results

Environment: All runs share one MacBook Pro (M4 Pro, 12 cores). At the start of RQ1 the 1-minute load average was 3.4 and machine-wide CPU use 9%; the production instance used under 1% CPU during our runs, and its scheduled security agents last ran *before* the experiments started (13:42 and 13:46

TABLE II
RQ1: VIOLATIONS PER WRITE (MEAN ± S.D., 5 SEEDS).

N	cond.	violations	after heal	cross-project	lockstep
1	no door	.217±.041	.000	.000	.05
1	door	**.000±.000**	.000	.000	1.00
2	no door	.289±.024	.106	.106	.05
2	door	**.000±.000**	.000	.000	1.00
5	no door	.307±.015	.118	.118	.05
5	door	**.000±.000**	.000	.000	1.00
10	no door	.308±.017	.108	.108	.05
10	door	**.000±.000**	.000	.000	1.00

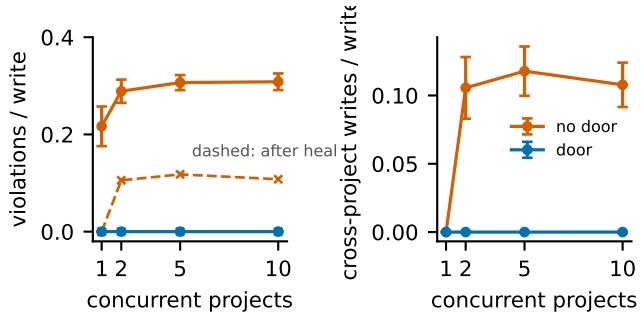


Fig. 2. RQ1. Left: violations per write, before (solid) and after (dashed) the audit heals. Right: writes landing in another project.

UTC; experiments ran from 14:30), so they did not overlap any measurement. Total model cost of the whole study: 5.72 USD.

A. RQ1: consistency at 1, 2, 5 and 10 concurrent projects

Table II and Fig. 2 summarise 40 trials (5 seeds per cell). Through the door, *no* write ever landed in a wrong folder or in another project, at any scale: the violation rate is 0 with zero variance for $N = 1$ to 10 (36 to 360 concurrent writes by 3 to 30 agents). Without the door, 22% of writes violate the schema at $N = 1$ and 29–31% once there are other projects to interfere with; the number of writes that land inside *another* project grows with N (from 0 to 11% of all writes). The audit and heal step repairs every wrong-folder write in the bypass condition, but *not* the cross-project writes: a file written into another project's valid code folder is structurally legal at rest, so 11–12% of writes remain wrong after healing. Only the write-time owner-scope guard prevents this class. Through the door, the registry mirrored the disk exactly before any healing (lockstep 1.0), against 0.05 without the door; healing restores 1.0 in both cases. The residual audit findings with the door (0.07–0.10 per write) are content-level issues the door cannot see (references to another project's file, duplicated utilities); all were healed to zero.

Cost of the door: The door is 10–19× slower per write than a raw file write, and its median per-write latency grows 1.5×, 3.3× and 6.4× from 1 to 2, 5 and 10 concurrent projects (Fig. 3), because every write opens its own database connection under contention. In absolute terms this remains small relative to an agent step, which takes seconds to minutes, but it is the first scaling bottleneck we observed.

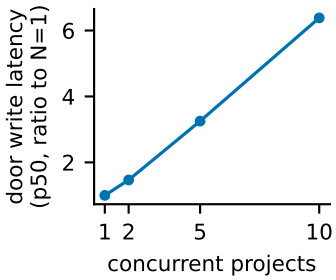


Fig. 3. RQ1. Median door write latency relative to one project.

TABLE III
RQ2: REAL LLM AGENTS, 3 REPETITIONS PER N (54 TASKS).

N	tasks	success	own code slot	cross-proj. files	cost/task ratio
1	3	1.00	.67	0	1.00
2	6	1.00	.50	0	0.99
5	15	.93	.40	0	1.18
10	30	.97	.17	0	1.13

B. RQ2: real LLM agents running concurrently

We ran 54 project tasks (1, 2, 5 and 10 concurrent projects, three repetitions; Table III). Task success, measured by running each produced test ourselves, was 100%, 100%, 93% and 97% (52/54 overall). No agent wrote into another project's folder (0 cross-project files). However, agents frequently wrote *outside the creation door*: only 67%, 50%, 40% and 17% of tasks placed their artefacts in their own project's code folder; the rest wrote to a top-level folder of the root, the agent's working directory. The audit flagged every such stray top-level path (flag ratio 1.0 in every run that had one), but does not move them automatically. Two consequences appeared at scale. In repetition 0 at $N = 10$ five task names were reused, and four pairs of projects overwrote each other's files in the shared stray folder (8 of 10 projects affected), an interference the in-workspace check could not see. In repetition 2, two agents followed the absolute repository path in the system's instructions and wrote into the main workspace instead of their throw-away root. Cost per task stayed flat as N grew (ratio to $N = 1$: 0.99, 1.18, 1.13), and the share of wall time with at least one model call in flight rose from 0.84 to 0.98, so the orchestration itself is not the cost driver; the server process tree used about 0.1% of the machine. Concurrency increased throughput only modestly: ten concurrent projects finished 1.4× more tasks per unit of wall time than one.

C. RQ3: the owner's own tests

The owner's lifecycle test (create four projects of different shapes through the door, monitor, delete) was *all green* in 3/3 runs: each creation had zero placement errors and zero violations, the deploy-shaped project reused the shared deploy capability through a pointer (no second deploy script), the multi-agent project linked three existing agents, and deletion returned every registry count to its exact baseline. The owner's adversarial test, which injects every violation type and requires

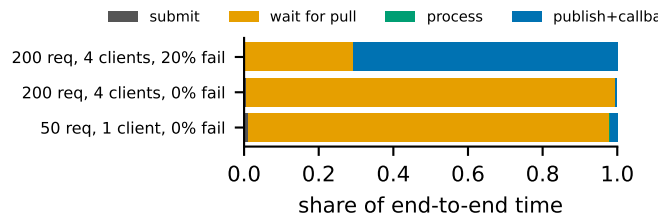


Fig. 4. RQ4. Share of end-to-end request time per stage (3 repetitions).

the system to correct it on disk and in the database, passed 32/32 checks in 3/3 runs. It also *failed* once, correctly: before we noticed the two escaped files of RQ2 in the main workspace, it reported them as cross-project duplicates and refused to declare a clean baseline.

Two LLM-driven owner tests revealed gaps. The multi-agent "Revealer" build (run with local deployment, since cloud access was blocked) stopped at its own pre-flight gate without spending tokens: the most recently added role (EasyPost) has no resources-and-dependencies section in the role catalogue. The owner's prompt journey (three prompt-created projects) planned all three, showed each team on every surface, and found zero violations and zero cross-owner references, but the derived team covered the required capabilities in only 1/3 projects (the new EasyPost role was attached to unrelated teams), output was produced in 2/3, and deletion left one or two registry file rows behind in 3/3. These are real findings that the system's own tests surface; we report them without changing the system.

D. RQ4: queue, callbacks and live sync

Fig. 4 shows where end-to-end time goes for a request through the Anchor point with zero processing time. Waiting in the queue for the next pull is 97–99% of the time; submission, publication and callback together are 1–3%. Four concurrent clients give 2.1× the throughput of one. With 20% of callbacks failing, the Anchor point's own retry sweep delivered 199/200 within the run and the last one at the next sweep (an at-least-once delivery, seen as a duplicate in the following run), so eventual callback success was 100% in all three repetitions, at the price of retry waits dominating (66–76% of time). For live sync, a pushed update was visible to a delta reader in under 0.1% of the poller's push interval; after a restart the mirror and all 20 sequenced messages were present, and a delta request from sequence 15 returned exactly 16–20.

VI. Discussion

Prevention and detection are both needed: RQ1 separates two mechanisms that are often conflated. The audit can repair everything that is wrong *in form* (misplaced files, unindexed rows, duplicates, foreign references), but a write into another project's legitimate folder is wrong only *in ownership*, and at rest it looks legal; 11–12% of writes stay wrong after healing without the door. The owner-scope guard at write time is what keeps interference at zero as projects are added. This mirrors the history in Table I: the audit came first, and interference between projects forced the door.

The weak point is the agent's own pen: RQ2 shows where consistency is not yet by construction: files the coding agent writes with its own tools do not pass through the door. Placement correctness fell from 67% to 17% as projects were added, and at ten projects agents overwrote each other in a shared stray folder. The audit saw every stray path, but flagging is not preventing. The fix follows the system's own design and is written as a proposal (Section VI-A): start each project's agents in their own folder and route the agent's write tool through the same owner-scope rule.

Consistency did not cost scale: Through the door the violation rate stayed at zero from 1 to 10 projects, cost per task stayed flat, and the system's own process tree was negligible next to model time. The measurable price is per-write latency under contention (6.4× at ten projects), which a connection pool would address without touching the schema.

Pulled queues trade latency for reachability: Almost all of a request's time is spent waiting to be pulled. That is the deliberate cost of never opening an inbound port on the owner's machine; callbacks with retries make the contract reliable, and receivers must tolerate an occasional duplicate.

Owner tests as instruments: The owner's own tests were the most sensitive instruments we had: they caught the two escaped files, an incompletely registered new role, and leftover registry rows, each without being designed for our study.

A. Proposals (not applied)

Following the owner's rule that no change may harm structure, consistency, database design or connections, these are written proposals only. They were each checked against that rule and would be regression-tested before any approval: (P1) move the terminal bridge's runtime files outside the version-controlled folder; (P2) route agents' own writes through the owner-scope rule and start agents in their project folder; (P3) pool database connections in the door; (P4) for *finding* reusable scripts only, add a sentence-embedding similarity as a second ranking signal next to keywords, while structural enforcement stays exact and rule-based; to be benchmarked on owner-labelled requests; (P5) a retrieval pipeline (recursive or semantic chunking, small embedding model, local vector index, top-k with cross-encoder re-ranking) for the one project that writes stories from large, stable reference PDFs, and nowhere else; (P6) send the request identifier as an idempotency key with every callback; (P7) complete the new role's catalogue entry.

VII. Threats to Validity

Construct: Our consistency metrics are the system's own checks (schema placement, owner scope, foreign references, registry lockstep) plus an authorship marker for cross-project writes; violations outside these classes are not counted. Task success for the LLM runs is a module plus a passing test that *we* execute, not the agents' claims.

Internal: The scripted workload (RQ1) replays the error classes observed in the system's history with fixed proportions; real agents may produce other mixes, which RQ2 partly addresses. All runs used the same machine while the owner's production instance and scheduled security agents were active; we record their load and report ratios rather than absolute times. Cloud and remote tools were blocked by design, so deployment-to-VM steps of the owner's tests were run locally (Section IV). The registry database is a private copy restored from a read-only dump.

External: This is one system, built mainly by one engineer with agents, on one laptop. Results may not transfer to other organisations; the mechanisms (one door, registry-only database, audit) are, however, framework-independent.

Reliability: Repeated runs (5 seeds for RQ1, 3 repetitions for RQ3 and RQ4, repeated LLM trials for RQ2) and a replication package with all scripts and raw results are provided.

VIII. Related Work

Why multi-agent systems fail: The MAST study analyses execution traces of seven multi-agent frameworks and attributes failures to specification issues, inter-agent misalignment and task verification [1]; it argues that many failures are organisational rather than model limitations. Odin is a practitioner's answer to the organisational part: structure is enforced by construction rather than requested in prompts. Multi-agent frameworks such as AutoGen, MetaGPT and ChatDev [6]–[8] coordinate roles within one task; they do not govern a shared, multi-owner workspace.

Reuse of agent skills (F1–F3, F7): Skill and workflow libraries let agents reuse code they wrote before [9]–[12]; SkillOps manages such a library as a self-maintaining ecosystem [13]; tool retrieval selects among many APIs [14]; plan and semantic caches reuse earlier answers [15]–[17]; case-based reasoning frames reuse in general [18], [19]. Odin's base-plus-delta discipline is the component idea [2], [3] and delta-oriented programming [4], re-discovered for agent scripts; what it adds is a single creation door and a database that is the only registry across *many owners' projects*.

Routing (F2): Model routers trade cost for quality [20]; Odin routes between reuse, delta and new work, with a deterministic floor under a semantic classifier [21], [22].

Sync, queueing and callbacks (F4–F6): Local-first software and CRDTs give strong convergence guarantees [23]; Odin uses the simpler combination of a durable mirror and per-scope sequence numbers in the spirit of logical clocks [24]. Its pulled queue is a minimal message broker [25], and its callbacks follow industry webhook practice with retries until acknowledgement [5].

Deployment and CI (F8–F10): Zero-trust access [26] removes reliance on network location; Odin's dynamic-IP firewall instead keeps a single current address allowed for operators without a static IP. Change-based deployment relates to regression test selection by file dependencies [27], and the self-healing deploy agent to LLM-based repair agents [28].

Conformance and guardrails (F11): Reflexion models check source against an intended architecture [29]; Odin's audit applies the same idea to the artefacts that agents create, and heals deviations. Runtime enforcement for agents [30], programmable rails [31] and capability-based designs [32], [33] constrain actions; the truth gate addresses the verification failures reported by MAST.

IX. Conclusion

A multi-project agentic ecosystem stays consistent as it grows when consistency is a property of the architecture rather than a request to the model. In Odin, one creation door with an owner-scope guard, a database that is the only registry, and an audit that heals at rest kept violations at zero from one to ten concurrent projects. Removing the door produced interference that no later audit could undo. The remaining gap is just as clear: the agents' own write tools sit outside the door. Closing it, as proposed, is the next step. Every component in Odin exists because something broke; we hope the failure timeline is as useful to other builders as the components themselves.

Acknowledgment

The author thanks the CEO and Founder for permission to describe the system. Publication requires the CEO/Founder's approval.

References

[1] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025, arXiv:2503.13657.

[2] Meta Open Source, "React: Thinking in components," https://react.dev/learn/thinking-in-react, accessed 2026-10-06.

[3] M. D. McIlroy, "Mass produced software components," in *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee*, 1968, pp. 138–155.

[4] I. Schaefer, L. Bettini, V. Bono, F. Damiani, and N. Tanzarella, "Delta-oriented programming of software product lines," in *Software Product Lines: Going Beyond (SPLC)*, ser. LNCS, vol. 6287. Springer, 2010, pp. 77–91.

[5] Stripe, "Receive stripe events in your webhook endpoint (retries and idempotency)," https://docs.stripe.com/webhooks, accessed 2026-10-06.

[6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[7] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[9] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[10] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," *arXiv preprint arXiv:2409.07429*, 2024.

[11] Z. Z. Wang, A. Gandhi, G. Neubig, and D. Fried, "Inducing programmatic skills for agentic tasks," *arXiv preprint arXiv:2504.06821*, 2025.

[12] S. Yu, G. Li, W. Shi, and P. Qi, "PolySkill: Learning generalizable skills through polymorphic abstraction for continual learning," in *International Conference on Learning Representations (ICLR)*, 2026.

[13] H. Pu, X. Song, and L. Zhao, "SkillOps: Managing LLM agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.

[14] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[15] Q. Zhang, M. Wornow, G. Wan, and K. Olukotun, "Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[16] B. Fu and D. Feng, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023, pp. 212–218.

[17] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025.

[18] A. Aamodt and E. Plaza, "Case-based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.

[19] K. Hatalis, D. Christou, and V. Kondapalli, "Review of case-based reasoning for LLM agents: Theoretical foundations, architectural components, and cognitive integration," *arXiv preprint arXiv:2504.06943*, 2025.

[20] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs from preference data," in *International Conference on Learning Representations (ICLR)*, 2025.

[21] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[22] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[23] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.

[24] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[25] S. Vinoski, "Advanced message queuing protocol," *IEEE Internet Computing*, vol. 10, no. 6, pp. 87–89, 2006.

[26] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," *;login: The USENIX Magazine*, vol. 39, no. 6, pp. 6–11, 2014.

[27] M. Gligoric, L. Eloussi, and D. Marinov, "Practical regression test selection with dynamic file dependencies," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2015, pp. 211–222.

[28] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[29] G. C. Murphy, D. Notkin, and K. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*, 1995, pp. 18–28.

[30] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026.

[31] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.

[32] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026.

[33] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.